

Amlogic_NPU_LLM_User_Guide

Revision: v0.3

Release Date: 2026-08-24

Copyright

© 2026 Amlogic. All rights reserved. No part of this document may be reproduced, transmitted, transcribed, or translated into any language in any form or by any means without the written permission of Amlogic.

Trademarks

 **Amlogic**, and other Amlogic icons are trademarks of Amlogic companies. All other trademarks and registered trademarks are property of their respective holders.

Disclaimer

Amlogic may make improvements and/or changes in this document or in the product described in this document at any time and without notice.

This product is not intended for use in medical, life saving, or life sustaining applications, nor in applications where failure or malfunction of an Amlogic product can reasonably be expected to result in personal injury, death or severe property or environmental damage.

Circuit diagrams and other information relating to products of Amlogic are included as a means of illustrating typical applications. Consequently, complete information sufficient for production design is not necessarily given. Though every effort has been made to ensure that this document is current and accurate, Amlogic makes no representations or warranties with respect to the accuracy or completeness of the contents presented in this document.

Amlogic products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Amlogic accepts no liability for any vulnerability.

Use of the materials may require a license of intellectual property from a third party or from Amlogic. This document conveys no express or implied licenses to any intellectual property rights belonging to Amlogic or any other party. Any links to third-party websites included in this document are for your convenience only. Amlogic does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Amlogic shall in no event be liable for any claims relating to or arising out of this document or any use or inability to use hereof.

Contact Information

- Website: www.amlogic.com
- Pre-sales consultation: contact@amlogic.com
- Technical support: support@amlogic.com

Revision history

Version	Change Description
V0.1	This is the First Version.
v0.2	Remove dependency on Tokenizer file
v0.3(2026-08-24)	Added support for VLM

Contents

1. Amlogic LLM Introduction

- 1.1 LLMSDK Architecture and Workflow
- 1.2 Introduction to the Amlogic LLM ToolChain
- 1.3 Supported Hardware Platforms and Model Support
- 1.4 Keyword Definitions

2. Preparing the Development Environment

- 2.1 Development Environment Verification
 - 2.1.1 Platform Type Verification
 - 2.1.2 DDK Version Verification
 - 2.1.3 System Environment Verification
 - 2.1.4 Device DDR Memory Capacity Verification
 - 2.1.5 Choosing an Integration Method
- 2.2 Toolkit Installation
 - 2.2.1 Installation Overview
 - 2.2.2 Toolkit Installation
 - 2.2.2.1 Setting Up a Virtual Environment
 - 2.2.2.2 Installing amlnn_toolkit
 - 2.2.2.3 Installing amlllm_edge_toolkit_lite
 - 2.2.3 Native C Build Environment Installation

3. Model Conversion

- 3.1 Conversion Overview
- 3.2 Exporting GGUF-Format Models
 - 3.2.1 Downloading Hugging Face Models
 - 3.2.2 Installing llama.cpp
 - 3.2.3 Converting Hugging Face to GGUF
- 3.3 Converting GGUF Models to ADLA
 - 3.3.1 Conversion Examples
 - 3.3.1.1 LLM Conversion Example
 - 3.3.1.2 VLM Conversion Example
 - 3.3.2 API Descriptions
 - 3.3.2.1 AMLNN Initialization
 - 3.3.2.2 Model Loading
 - 3.3.2.3 Configuring Conversion Parameters
 - 3.3.2.4 Model Building
 - 3.3.2.5 Resource Release

4. On-Device Inference (Python API)

- 4.1 Workflow Overview
- 4.2 Python Demo Examples
 - 4.2.1 LLM Example
 - 4.2.2 VLM Example
- 4.3 Python API Reference
 - 4.3.1 Initialization
 - 4.3.2 `setLogLevel`
 - 4.3.3 `config`
 - 4.3.4 `init`
 - 4.3.5 `run`
 - 4.3.6 `reset_session`
 - 4.3.7 `break_generation`
 - 4.3.8 `set_chat_template`
 - 4.3.9 `uninit`

5. On-Device Inference (C++ API)

5.1 Workflow Overview

5.2 Core Data Structures and Enums

5.2.1 LLMContext

5.2.2 AML_LLMRunStatus

5.2.3 AML_LLMRetStatus

5.2.4 AML_LLMInputType

5.2.5 AML_LLMRunMode

5.2.6 AML_LLMIImageType

5.2.7 AML_LLMSamplingMode

5.2.8 AML_LLMLInitConfig

5.2.9 AML_LLMLInitExtend

5.2.10 AML_LLMLRunConfig

5.2.11 AML_LLMLRunExtend

5.2.12 AML_LLMLImageInput

5.2.13 AML_LLMMultimodalInput

5.2.14 AML_LLMTokenInput

5.2.15 AML_LLMLInput

5.2.16 AML_LLMLGenerationResult

5.2.17 AML_LLMLEmbeddingResult

5.2.18 AML_LLMLResult

5.3 Basic API Reference

5.3.1 LLMResultCallback

5.3.2 aml_llm_init

5.3.3 aml_llm_run

5.3.4 aml_llm_uninit

5.3.5 aml_llm_reset

5.3.6 aml_llm_break

5.3.7 aml_llm_set_chat_template

5.4 Error Return Codes

5.5 Multimodal Notes

5.5.1 Multimodal Call Flow

5.5.2 VLM Example

6. FAQ and Performance Reference

6.1 Performance Reference Data

6.2 Common Error Troubleshooting

6.3 Reference Links

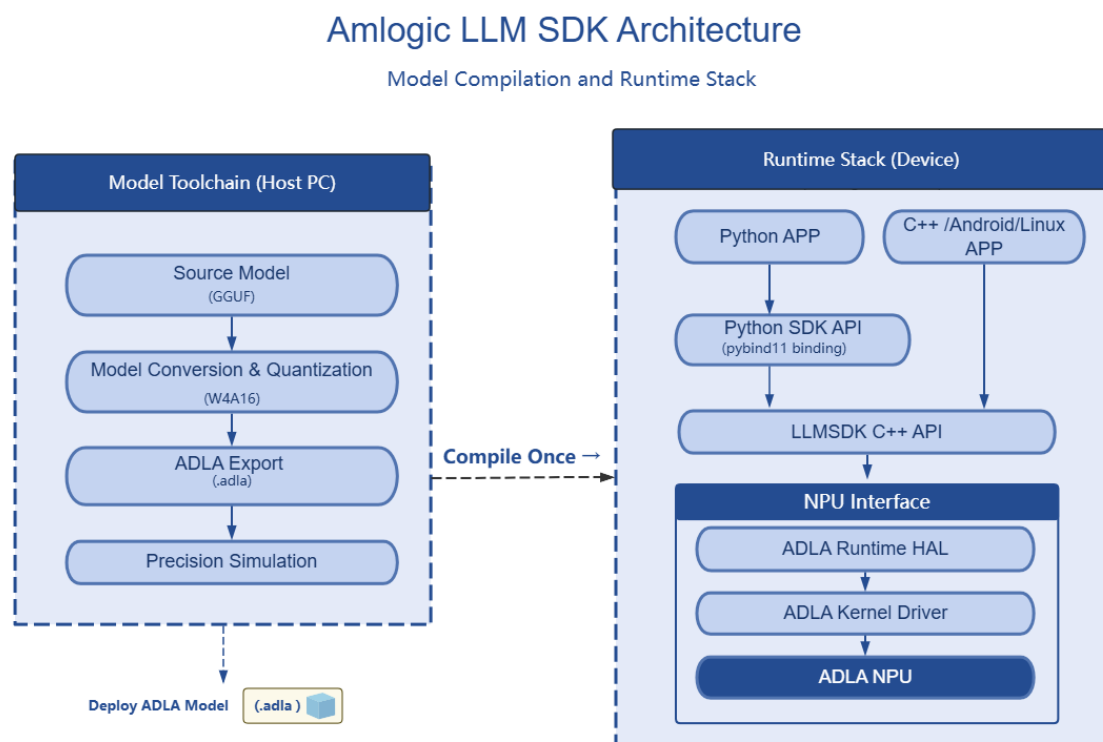
1. Amlogic LLM Introduction

Amlogic LLM is a one-stop software solution designed for on-device deployment of Large Language Models (LLMs). Its primary objective is to lower the integration barrier of mainstream foundation models on the Amlogic NPU platform. The solution covers the entire workflow, from model conversion to edge-side inference, and consists of two core components: **amlInn-toolkit** and **Runtime**.

- **amlInn-toolkit**: This PC-based model conversion toolkit supports LLM/VLM model conversion, quantization, and ADLA export.
- **Runtime**: A highly efficient edge-side inference execution layer that abstracts low-level NPU hardware details and provides two levels of integration
 - **Python Runtime (amllm-edge-toolkit-lite)**: A lightweight Python package with a minimal API, suitable for rapid validation and application prototyping.
 - **C++ Runtime (libllmsdk.so)**: A native runtime interface for Android and Linux platforms, supporting deep system integration and advanced performance tuning.

1.1 LLMSDK Architecture and Workflow

The software stack architecture is shown below:



As shown in the figure, LLMSDK adopts a decoupled "compile once, run on multiple devices" architecture, consisting of two core components: amlInn-toolkit and Runtime:

- **Development & conversion side:** The user feeds in a raw model in GGUF or similar format; amlInn-toolkit performs conversion, quantization, and ADLA export to generate ADLA model files deeply optimized for the Amlogic NPU.
- **Device inference side:** After the ADLA model is deployed on Amlogic hardware, both the Python and C++ inference paths are uniformly scheduled through the NN HAL layer, invoking NPU hardware resources to achieve low-power, high-efficiency on-device inference.

This architecture greatly simplifies the migration path of large models from the cloud to edge devices and flexibly adapts to the deployment requirements of different development scenarios.

1.2 Introduction to the Amlogic LLM ToolChain

The ToolChain is split into two stages: "development & conversion" and "on-board inference":

- **Conversion stage (x86 PC side):** `am1nn_toolkit` (X86-architecture whl package) only accepts GGUF-format LLM/VLM model input at f16/bf16/f32 precision, and by default uses the Q4A16 quantization scheme (Weight int4 group32, Activation f16). The full quantization and ADLA model export flow can be completed via the Python interface.
- **Inference stage (Board-side Python):** `am11lm_edge_toolkit_lite` (aarch64-architecture whl package) is deployed on the edge device. It directly loads the converted ADLA model through the Python API to perform LLM/VLM on-device inference.
- **Inference stage (llm_runtime library):** A dedicated C++ inference SDK for large language model scenarios. Integration only requires linking `lib11msdk.so` and including the `11msdk.h` header file. It fully supports Android and Linux (Buildroot/Yocto/Debian) platforms and provides LLM-specific core capabilities such as model loading, inference scheduling, and context management.

1.3 Supported Hardware Platforms and Model Support

- **Supported hardware platforms**
 - C305X2/A123X
 - A311Y3
- **Verified supported model list**
 - The SDK has completed adaptation and validation of the following mainstream open-source models. Developers can obtain the corresponding models and deploy them on-device:

Category	Model series	Specific model
LLM	DeepSeek	DeepSeek-R1-Distill-Qwen-1.5B
LLM	Qwen	Qwen-1.8B-Chat ; Qwen1.5-0.5B-Chat ; Qwen1.5-1.8B-Chat ; Qwen2-0.5B-Instruct ; Qwen2-1.5B-Instruct ; Qwen2.5-0.5B-Instruct ; Qwen2.5-1.5B-Instruct ; Qwen2.5-3B-Instruct ; Qwen3-0.6B ; Qwen3-1.7B ; Qwen3-4B-instruct-2507
LLM	Gemma	gemma-2b-it ; gemma-2-2b-it ; gemma-3-270m-it ; gemma-3-1b-it ; gemma-4-E2B-it ; gemma-4-E4B-it
LLM	TinyLlama	TinyLlama-1.1B-Chat-v0.4 ; TinyLlama-1.1B-Chat-v1.0
LLM	Llama	Llama-3.1-8B-Instruct ; Llama-3.2-1B-Instruct ; Llama-3.2-3B-Instruct

Category	Model series	Specific model
LLM	Phi	phi-1.5 ; phi-2 ; Phi-3.5-mini-instruct
LLM	ChatGLM	chatglm3-6b
VLM	Qwen-VL	Qwen2-VL-2B-Instruct ; Qwen2.5-VL-3B-Instruct ; Qwen3-VL-2B-Instruct ; Qwen3-VL-4B-Instruct
VLM	InternVL	InternVL2-1B ; InternVL3-1B-Instruct
VLM	MiniCPM	MiniCPM3-4B

Note: The `gemma-4-E2B-it` and `gemma-4-E4B-it` models are currently only supported for deployment using the pre-compiled ADLA format provided by Amlogic. The current version of `am1nn_toolkit` does not yet support conversion of these two models.

1.4 Keyword Definitions

- **ADLA:** ADLA (Amlogic Deep Learning Accelerator) refers to model files that run on the Amlogic platform, with the `.adla` extension.
- **Armbian (Debian):** The Debian system, uniformly abbreviated as Armbian throughout the remainder of this document.

2. Preparing the Development Environment

2.1 Development Environment Verification

2.1.1 Platform Type Verification

The current LLM solution only supports the A311Y3 and C305X2/A123X Amlogic NPU platforms; other platforms in the same series are not yet supported. Before starting the model conversion workflow, platform compliance verification must be completed to confirm that the target hardware is within the officially supported range, thereby preventing subsequent deployment failures caused by incompatible scenarios at the source.

When performing ADLA model conversion, you need to obtain the `target_platform` parameter that exactly matches the target hardware, so as to avoid conversion failure or inference anomalies due to parameter mismatch. Platform type verification can be completed in two independent ways; all verification results can be checked against the **Part_number/Serial/target_platform correspondence table** to obtain the final, accurate `target_platform` value.

Part_number	Serial	target_platform
A311Y3	Serial: 4d0a 01020000000070e84a	PRODUCT_PID0XA007
C305X2/A123X	Serial: 4f0a 01010000000054c6d1	PRODUCT_PID0XA008

Method 1: Quick verification via Part_number

Directly read the Part_number marking engraved on the board hardware and match the read number against the correspondence table to obtain the correct `target_platform` parameter for that board.

Method 2: Verification via Serial number

Obtain the Serial information: After the board has booted normally, connect to the device via ADB and execute the following commands to read the Serial number:

```
adb shell cat /proc/cpuinfo

# or

adb shell cat /proc/cpu_chipid
```

Matching rules:

- **Preferred rule:** Take the **first 4 characters** of the Serial number and match them against the correspondence table to obtain the target `target_platform` value.
- **Fallback rule:** If no match is found with the first 4 characters, fall back to matching on the **first 3 characters** of the Serial number to finally obtain the correct `target_platform` parameter.

2.1.2 DDK Version Verification

First confirm that the NPU module has been loaded successfully, and confirm the loaded NPU Driver version.

- **Driver loading verification:** Connect to the board via adb shell or use the board's own command window.
 - **Loading verification:** Execute `lsmod | grep adla_core`; the result should contain `adla_core`, as shown below:

```
t7 an400:/ $ lsmod | grep adla_core
adla_core                221184  0
amlogic_cpuinto         16384  18 amvdec_av1_t5d_v41, amvdec_
```

- **Permission verification:** Execute `ls -l /dev/adla0`; the permissions should be `666`, as shown below:

```
t7 an400:/ $ ls -l /dev/adla0
crw-rw-rw- 1 system system 463, 0 1970-01-01 00:00 /dev/adla0
```

- **DDK version verification:**
 - **Driver version verification:** Reboot the board, connect via the serial port or an adb shell in a CMD window, and execute `dmesg | grep adla` to obtain a log similar to the following:

```
5.405204] adla kmd version: 0.10.7.1
```

NPU Driver version: 0.10.7 (just take the version number corresponding to the first three segments)

- **C Runtime version verification:** The board supports the following verification commands:

```
# Command for Android 32-bit strings /vendor/lib32/libllmsdk.so | grep "LLMSDK"

# Command for Android 64-bit
strings /vendor/lib64/libllmsdk.so | grep "LLMSDK"

# Command for Linux (Buildroot 32 / Buildroot 64 / Yocto 32 / Yocto 64 /
Debian 64 / Ubuntu 64)
strings /usr/lib/libllmsdk.so | grep "LLMSDK"

# Expected output example: LLMSDK, v1.x.x, 202x.0x
```

- **DDK version verification:**

The corresponding DDK version must be determined based on the NPU Driver version, and the C Runtime and amlnn-toolkit versions must be matched accordingly.

DDK version	NPU Driver version	C Runtime Version	amlInn-toolkit version
DDK5.10.4.10	2.0.0		amlInn_toolkit_v1.x.x

Note: The contents of this document support NPU Driver versions >= 2.0.0.

2.1.3 System Environment Verification

This section is used to verify the board-side system environment and hardware architecture, providing an accurate basis for selecting the appropriate version of the **cross-compilation toolchain** and determining the integration approach in later chapters. If the customer is already clear about the current platform environment, this chapter can be skipped; if the platform information is uncertain, please complete the environment verification according to the following steps.

Step 1: First determine whether it is an Android or Linux platform

Command: `adb shell getprop ro.product.cpu.abi`

Decision logic:

- **Output** `arm64-v8a` : confirmed to be a 64-bit Android system
- **Output** `armeabi-v7a` : confirmed to be a 32-bit Android system
- **Error** `command not found` : determined to be a Linux system; proceed to Step 2

Step 2: Further verification of the Linux system (Buildroot / Yocto / Armbian) environment

Step 2.1: Determine whether it is an Armbian / Yocto system

Command: `cat /version.txt`

Example reference output (using Armbian as an example)

```

imagename:a9-by401-bookworm-20260707030949 BRANCH=
VERSION=Linux/Debian/Armbian-PB#786
SPIN=Linux/Debian/Armbian-PB#786
BUILD_TIME="2026-07-07 03:27:41"
Generated on Tue Jul 7 03:27:41 UTC 2026

Compiler:
aarch64-none-linux-gnu-gcc (Arm GNU Toolchain 11.3.Rel1) 11.3.1 20220712
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Decision logic:

- **If the file exists:** it is determined to be an Armbian/Yocto system; then use the Compiler version information to determine whether it is a 32-bit or 64-bit system. The example above corresponds to an **Armbian 64-bit (AArch64)** system.
- **If the file does not exist:** the command fails, so it is determined to be a Buildroot system; proceed to Step 2.2.

Note: The above verification method applies only to Amlogic official Release version boards. For third-party development boards, please verify on your own whether the same version file and decision logic are supported.

Step 2.2: Determine the Buildroot system architecture

Command: `adb shell uname -m`

Decision logic:

- `aarch64` : confirmed to be a 64-bit Buildroot system
- `armv7l` : confirmed to be a 32-bit Buildroot system

2.1.4 Device DDR Memory Capacity Verification

Running LLM models places high demands on available DDR memory resources. You should first verify the device's actual DDR memory capacity in order to preliminarily determine the model size tier the device can carry. LLM models of different parameter scales require the device side to meet specific DDR memory reservation thresholds, ensuring that the NPU can complete model loading and run stably.

Execute: `free -m`

```
akee:/data # free -m
              total        used          free      shared    buffers
Mem:           7925         1029          6895           3           0
-/+ buffers/cache:      1029         6895
Swap:           599          359           240
```

The runtime memory consumption of currently supported models is shown in the table below. These values vary with context length (**Max Context**) and Compiler version iterations, and should only be used as a preliminary basis for model selection.

Model	Version	Param	Max Context	Memory (MB)
DeepSeek	R1	1.5B	2048	1268.62
Qwen	1	1.8B	2048	1634.78
Qwen	1.5	0.5B	2048	644.60
Qwen	1.5	1.8B	2048	1575.23
Qwen	2	0.5B	2048	402.92
Qwen	2	1.5B	2048	1075.24
Qwen	2.5	0.5B	2048	475.95
Qwen	2.5	1.5B	2048	1075.24
Qwen	2.5	3B	2048	2021.83
Qwen	3	0.6B	2048	779.52
Qwen	3	1.7B	2048	1541.24
Qwen	3	4B	2048	2842.98

Model	Version	Param	Max Context	Memory (MB)
Qwen	3	4B(Instruct)	2048	2842.98
TinyLlama	0.4	1.1B	2048	772.15
TinyLlama	1	1.1B	2048	772.06
Llama	2	7B	2048	5217.70
Llama	3.2	1B	2048	953.43
Llama	3.2	3B	2048	2347.69
Gemma	1	2B	2048	1836.43
Gemma	2	2B	2048	2130.33
Gemma	3	270M	2048	655.80
Gemma	3	1B	2048	1146.57
Gemma	4	E2B	2048	3542.01
Gemma	4	E4B	2048	5776.59
Phi	1.5	1.3B	2048	1404.81
Phi	2	2.7B	2048	2605.29
Phi	3	3.8B	2048	3400.62
ChatGLM3	3	6B	2048	4048.54
MiniCPM	3	4B	2048	4434.37
InternVL	2	1B	2048	526.52
InternVL	3	1B	2048	526.53
Qwen-VL	2	2B	2048	1315.36
Qwen-VL	2.5	3B	2048	2345.54
Qwen-VL	3	2B	2048	1704.13
Qwen-VL	3	4B	2048	3259.33

2.1.5 Choosing an Integration Method

- Supported integration methods:

	Buildroot	Android	Debian	Yocto	Dependency
Native(c/c++ Api)	✓	✓	✓	✓	amlnn_toolkit /ADLA LLMSDK
Native(python Api)	×	×	✓	✓	amlnn_toolkit /amlllm_edge_toolkit_lite

- **Integration method descriptions:**
 - Native (C/C++ API)
 - Use the toolkit to convert supported LLM models (GGUF) into ADLA models.
 - Use Amlogic's unified, encapsulated LLMSDK API to load and run inference on ADLA models.
 - Native (Python API)
 - Use the toolkit to convert supported LLM models (GGUF) into ADLA models.
 - Then use the Python API of amlllm_edge_toolkit_lite to load and run inference on the ADLA model.
- **API comparison:**

Feature	Python Runtime (amlllm-edge-toolkit-lite)	C++ Runtime (libllmsdk.so)
Language	Python package, concise API	Native C++ API, supports deep system integration
Performance/efficiency	Good performance; calls the C++ Runtime underneath	Ultimate performance, suitable for performance-sensitive scenarios
Development	Fast, convenient for prototyping and functional validation	Relatively slow; integration and compilation are complex
Target platforms	Linux (aarch64, 如 Armbian/Yocto)	Android 和 Linux (full platform support)
Deployment environment	Python environment, simple dependencies	Requires NDK/cross-compilation toolchain and <code>libllmsdk.so</code>

- **API selection recommendations:**

	Applicable users / scenarios
Python Runtime	Users focused on rapid development and validation: pursue the fastest development speed and need to quickly validate model behavior; are developing Python-based applications, scripts, or services; do not have extreme low-level performance requirements; and deploy only on Linux devices.
C++ Runtime	Performance-sensitive users pursuing deep integration: need to deeply integrate LLM functionality into native Android or Linux system applications (e.g., system-level services, apps); pursue the lowest latency and highest inference performance; and need to deploy on Android or in non-Python environments.

2.2 Toolkit Installation

2.2.1 Installation Overview

Different board system environments support different integration methods, and different integration environments require different tool chains. First determine the integration method based on the system environment, then install the corresponding Toolkit.

	amlnn_toolkit	amlllm_edge_toolkit_lite	Native C build environment
Native (C/C++ API)	✓	×	✓
Native (Python API)	✓	✓	×

2.2.2 Toolkit Installation

After selecting the integration method, refer to this chapter for Toolkit installation instructions. It is recommended to use Miniforge to manage the Python environment on both the PC side and the Debian device side.

2.2.2.1 Setting Up a Virtual Environment

- **Download the installer:**
 - x86_64: `wget https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-x86_64.sh`
 - aarch64: `wget https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-aarch64.sh`
- **Install Miniforge:**
 - `chmod +x Miniforge3-Linux-*.sh`
 - x86_64: `bash Miniforge3-Linux-x86_64.sh`
 - aarch64: `bash Miniforge3-Linux-aarch64.sh`
- **Verify the installation:**
 - Run `conda config --show channels` successfully; the result should contain `- conda-forge`
- **Create the virtual environment**
 - Create: `conda create -n amlnn_toolkit python=3.10`
 - Activate: `conda activate amlnn_toolkit`
 - The Python version can be adjusted as needed (3.10 recommended); 'amlnn_toolkit' is the virtual environment name and can be changed at will.

2.2.2.2 Installing amlnn_toolkit

`amlnn_toolkit` is a model conversion toolkit installed on the PC side. To install it, first activate the corresponding Conda environment, and then install the toolkit using the local wheel package. If you are using a precompiled ADLA model directly (see Section 3.1 for details), you may skip the installation of this toolkit.

Install:

Select the corresponding requirements and whl files based on the Python version and processor architecture.

Download link: https://github.com/Amlogic-NN/amlnn-toolkit/tree/main/amlnn_toolkit/whl/linux_x86

#for amlnn_toolkit

Step 1: Install dependencies one by one (to avoid version conflicts)

for req in \$(cat requirements_x86_64.txt); do pip install \$req; done

Step 2: Install the toolkit whl package

toolkit-x.x.x is the toolkit version; cpxx is the Python version tag (e.g., cp310, cp311)

pip install amlnn_toolkit-x.x.x-cpxxx-cpxxx-linux_x86_64.whl

#Note: Do NOT use `pip install -r requirements\_x86\_64.txt`, as it causes dependency version conflicts. Dependencies must be installed one by one.

Verify:

Verify toolkit-x.x.x version

python -c "from amlnn.api import AMLNN_LLM; print('amlnn_toolkit import OK')"

2.2.2.3 Installing amllm_edge_toolkit_lite

After activating the `amlnn_toolkit` Conda environment, install via the local wheel package. Note that `amllm_edge_toolkit_lite` is a model inference toolkit designed specifically for board-side Yocto/Armbian system environments, used for edge deployment.

Install:

Download link: https://github.com/Amlogic-NN/amlnn-toolkit/tree/main/amlnn_toolkit_lite/amllm_edge_toolkit_lite/whl/aarch64

#for amlnn_edge_toolkit_lite

pip install -r requirements.txt

pip install amllm_edge_toolkit_lite-1.x.x-cp310-cp310-linux_aarch64.whl

Verify:

Verify toolkit_lite-x.x.x version

python -c "from amllmlite.api import AMLLMLite; print('amllm_edge_toolkit_lite import OK')"

2.2.3 Native C Build Environment Installation

Native C build environment installation means installing, on the PC side, the cross-compilation tools corresponding to the device system to be built. Different board system environments require different toolchains, as follows:

- **Android System:**

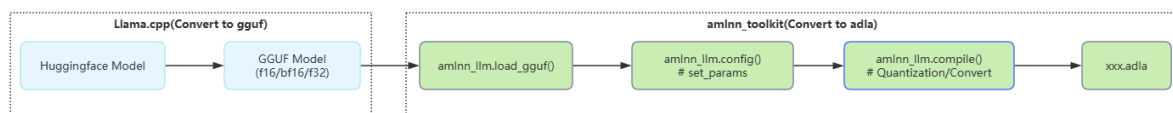
- Cross-compilation tool download link: [Android NDK download](#); android-ndk-r25c-linux.zip is recommended.

- Extract to a directory, e.g., /opt/android-ndk-r25c
- Build configuration
 - When building, set `ANDROID_NDK_PATH` in `build-android64.sh` / `build-android32.sh` to the local toolchain path.
- **Buildroot System:**
 - Cross-compilation tool download links:
 - 32-bit: [Buildroot 32 toolchain download](#)
 - 64-bit: [Buildroot 64 toolchain download](#)
 - Extract to a directory, e.g.:
 - 32-bit: /opt/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi
 - 64-bit: /opt/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu
 - Build configuration
 - When building, set `BUILDROOT_TOOLCHAIN_PATH` in `build-buildroot64.sh` / `build-buildroot32.sh` to the local toolchain path.
- **Debian/Yocto System:**
 - Cross-compilation tool download links:
 - 32-bit: [32-bit toolchain download](#)
 - 64-bit: [64-bit toolchain download](#)
 - Execute the corresponding script:
 - 32-bit: `./poky-glibc-x86_64-amlogic-bsp-armv7at2hf-neon-mesons7-bh201-5.15-a32-toolchain-4.0.20.sh`
 - 64-bit: `./poky-glibc-x86_64-meta-toolchain-armv8a-mesont7-an400-5.15-a64-toolchain-4.0.20.sh`
 - Installation choices:
 - Choose the installation path, e.g., /opt/poky/4.0.20.
 - Choose to enter yes.
 - After a short wait, the build toolchain will be installed.
 - Build configuration
 - When building, set `YOCTO_TOOLCHAIN_PATH` in `build-yocto64.sh` / `build-yocto32.sh` to the local toolchain path.

3. Model Conversion

`amlnn-toolkit` provides model conversion functionality, supporting conversion of GGUF-format large language models into ADLA-format models that run natively on the Amlogic NPU. This section describes the complete conversion workflow of `amlnn-toolkit` for LLM models in detail, serving as an operational reference for users.

3.1 Conversion Overview



Currently, `amlnn-toolkit` only supports importing and converting LLM models in **GGUF** format. Therefore, before starting the conversion workflow, please make sure you have already obtained the GGUF-format file of the target model.

The preprocessing part in the figure above (i.e., the "Convert to GGUF" step) must be completed by the user. The final ADLA model can be obtained through the following three approaches:

1. Use pre-compiled ADLA models:

- Directly download pre-compiled ADLA model files from the official Amlogic-NN Hugging Face repository ([Amlogic-NN](#)). These can be directly deployed and run on edge boards, greatly shortening the deployment preparation cycle.

2. Download GGUF models and convert to ADLA:

- Directly download officially published, standard GGUF-format model files from open-source model communities such as Hugging Face.
- Import the files into the `am1nn-toolkit` tool to perform the ADLA conversion.

3. Convert to GGUF yourself, then convert to ADLA:

- Use the official open-source conversion tool provided by `llama.cpp` to convert the original Hugging Face-format native model into a standard GGUF-format model file.
- Import the file into the `am1nn-toolkit` tool to perform the ADLA conversion.

3.2 Exporting GGUF-Format Models

GGUF is an efficient binary model format led by the `llama.cpp` community, designed specifically for high-performance inference of large language models (LLMs) on edge and local environments.

- **If you already have a GGUF model:** If you have directly downloaded the GGUF-format file of the target model from communities such as Hugging Face, please **skip this chapter** and proceed directly to the subsequent model conversion or deployment steps.
- **If self-conversion is required:** This chapter explains in detail how to use the official tools provided by `llama.cpp` to convert the original Hugging Face-format model into standard GGUF format.

3.2.1 Downloading Hugging Face Models

Make sure the complete HF model files have been downloaded (including `.safetensors` or `.bin`, `tokenizer.json`, etc.).

Method 1: Command-line download

```
pip install huggingface_hub
pip install -U huggingface_hub[cli]
# Optional: Login to access gated model
huggingface-cli login
# Download the model (disable symlinks for compatibility)
huggingface-cli download Qwen/Qwen2.5-0.5B \
  --local-dir ./qwen2.5-0.5b --local-dir-use-symlinks False
```

Note: Downloads via this method may fail due to network issues; in that case, Method 2 is recommended.

Method 2: Web download

Open the link of the model in the supported model list, click **Files and versions**, then click the download link to the right of each file to download the original model files directly, as shown below:

The screenshot shows the Hugging Face interface for the Qwen2.5-0.5B-Instruct model. The 'Files and versions' tab is active, displaying a table of files. A blue box highlights the files: `config.json`, `generation_config.json`, `merges.txt`, `model.safetensors`, `tokenizer.json`, `tokenizer_config.json`, and `vocab.json`. Another blue box highlights the download icons for these files.

File	Size	Download Icon	Description
<code>.gitattributes</code>	1.52 kB	Download	initial commit
<code>LICENSE</code>	11.3 kB	Download	update README & LICENSE
<code>README.md</code>	4.92 kB	Download	Update metadata: Add lib
<code>config.json</code>	659 Bytes	Download	Upload folder using hugg
<code>generation_config.json</code>	242 Bytes	Download	Upload folder using hugg
<code>merges.txt</code>	1.67 MB	Download	Upload folder using hugg
<code>model.safetensors</code>	988 MB	Download	Upload folder using hugg
<code>tokenizer.json</code>	7.03 MB	Download	Upload folder using hugg
<code>tokenizer_config.json</code>	7.31 kB	Download	Upload ./tokenizer_conf
<code>vocab.json</code>	2.78 MB	Download	Upload folder using hugg

In general, converting Hugging Face to GGUF only requires downloading `config.json`, `safetensors` (or `pytorch_model.bin`), and `tokenizer.json` (or `tokenizer.model` or `xx.tiktoken`).

3.2.2 Installing llama.cpp

Converting from Hugging Face to GGUF is done with llama.cpp. You must first download the llama.cpp source code and install the dependencies, as follows:

```
conda create -n llama_cpp python=3.10
conda activate llama_cpp

git clone --depth 1 --branch b6199 https://github.com/ggerganov/llama.cpp.git
cd llama.cpp
pip install -r requirements.txt
```

3.2.3 Converting Hugging Face to GGUF

Using the `convert_hf_to_gguf.py` script provided by llama.cpp, models in Hugging Face format can be converted to GGUF format. The Qwen2.5-0.5B model is used below as an example:

```
# usage: python convert_hf_to_gguf.py [model_dir] --outfile [out_path]
python convert_hf_to_gguf.py \
    ./qwen2.5-0.5b \
    --outfile qwen2.5-0.5b_fp16.gguf \
    --outtype f16
```

Parameter descriptions:

Parameter/path	Description
./qwen2.5-0.5b	Input path: local folder path of the Qwen2.5-0.5B Hugging Face model.
--outfile	Output file: specifies the generated GGUF model file name (e.g., qwen2.5-0.5b_fp16.gguf).
--outtype	Precision type: specifies the weight precision of the output model; f16 means half-precision floating point.

Result:

After the command executes successfully, a model file named `qwen2.5-0.5b_fp16.gguf` will be generated in the current directory; this file can then be used in the subsequent `amlnn-toolkit` conversion flow.

3.3 Converting GGUF Models to ADLA

3.3.1 Conversion Examples

The conversion is performed through the interfaces provided by the `amlnn-toolkit` package and mainly consists of three steps: load, build, and export.

3.3.1.1 LLM Conversion Example

```
from amlnn.api import AMLNN_LLM

# AMLNN_LLM init
amlnn_llm = AMLNN_LLM(log_level='INFO')

modelpath="./Qwen2.5-0.5B-Instruct-F16.gguf"

# load gguf model
amlnn_llm.load_gguf(model=modelpath)

amlnn_llm.config(prefill_batch_length=32, max_context=2048,
transformer_model_type='LLM', target_platform='PRODUCT_PID0XA007',
output_dir="output")

amlnn_llm.compile()

amlnn_llm.uninit()
```

3.3.1.2 VLM Conversion Example

```
from amlnn.api import AMLNN_LLM

# AMLNN_LLM init
amlnn_llm = AMLNN_LLM(log_level='INFO')

#for VLM's vision model
modelpath="./mmpoj-F32.gguf"
```

```

amlnn_llm.load_gguf(model=modelpath)
amlnn_llm.config(transformer_model_type='MMPROJ',
target_platform='PRODUCT_PID0XA007', output_dir="output")
amlnn_llm.compile()
amlnn_llm.uninit()

#for VLM's LLM model
modelpath="./Qwen3-VL-4B-Instruct-BF16.gguf"
amlnn_llm.load_gguf(model=modelpath)
amlnn_llm.config(prefill_batch_length=32, max_context=2048,
transformer_model_type='VLM', target_platform='PRODUCT_PID0XA007',
output_dir="output")
amlnn_llm.compile()
amlnn_llm.uninit()

```

3.3.2 API Descriptions

3.3.2.1 AMLNN Initialization

When using the amlnn-toolkit API, first call AMLNN_LLM() to initialize the AMLNN_LLM object, and call uninit() at the end to release resources.

Parameter	Type	Description	Required
log_level	str	Log level setting, default: 'INFO' . Currently supports ['DEBUG', 'INFO', 'WARNING', 'ERROR', 'CRITICAL']	No

```

from amlnn.api import AMLNN_LLM
amlnn_llm = AMLNN_LLM(log_level='INFO')

...

amlnn_llm.uninit()

```

3.3.2.2 Model Loading

Function: Load a GGUF model.

Return value: 0: success, -1: failure

Parameter	Type	Description	Required
model	str	Path to the gguf model file	Yes

The function is defined as follows:

```

#example
modelpath="./Qwen2.5-0.5B-Instruct-F16.gguf"

amlnn_llm.load_gguf(model=modelpath)

```

- **Special conversion flow for VLM vision encoders**

When the Vision module (vision encoder) of a VLM model is not compatible with the GGUF format, direct use of the GGUF conversion path is prohibited; instead, switch to the ONNX conversion channel to produce ADLA-format output.

- **Guidance:**

- **Applicable scenarios:** only for cases where the Vision Encoder cannot be converted via the standard GGUF path. Typical example: the **FastVLM 0.5B vision module**.
- **Reference documentation:** follow the ONNX model conversion flow in 《03_Amlogic_NPU_API_Reference_AMLNN》
- **Complete conversion chain:** Vision module source model --> export to standard ONNX format --> ADLA

3.3.2.3 Configuring Conversion Parameters

Function: Configure model conversion parameters.

Return value: 0: success, -1: failure

Parameter	Type	Description	Required
prefill_batch_length	int	Controls the maximum number of tokens processed in a single prefill pass. Recommended values: [32, 64]. Default: 32. Effective when transformer_model_type is LLM/VLM.	No
max_context	int	Maximum context. Recommended values: [256, 512, 1024, 2048, 4096, 8192, 16384]. Default: 2048. Effective when transformer_model_type is LLM/VLM.	No
transformer_model_type	str	Valid values: ['MMPROJ', 'LLM', 'VLM']. Default: LLM. For a text-only LLM deployment, keep the default <code>LLM</code> . For a multimodal (VLM) deployment, conversion is done in two steps: set <code>MMPROJ</code> for the vision part and <code>VLM</code> for the language part.	Yes
target_platform	str	Platform type; supports <code>PRODUCT_PID0XA007</code> and <code>PRODUCT_PID0XA008</code> . Default: <code>PRODUCT_PID0XA007</code> . Obtain the correct value per Section 2.1.1: Platform Type Verification.	Yes
output_dir	str	Output path, default: None, meaning the default output path is: model_name + "_adla_unify"	No

```

#Example for LLM
amlnn_llm.config(prefill_batch_length=32, max_context=2048,
transformer_model_type='LLM', target_platform='PRODUCT_PID0XA007',
output_dir="output")

#Example for VLM's Vision
amlnn_llm.config(transformer_model_type='MMPROJ',
target_platform='PRODUCT_PID0XA007', output_dir="output")

#Example for VLM's LLM
amlnn_llm.config(prefill_batch_length=32, max_context=2048,
transformer_model_type='VLM', target_platform='PRODUCT_PID0XA007',
output_dir="output")

```

3.3.2.4 Model Building

Function: Compile the AMLLLM model and output the final adla files.

Return value: 0: success, -1: failure

```

#example
amlnn_llm.compile()

```

3.3.2.5 Resource Release

Function: Release runtime and model resources.

```

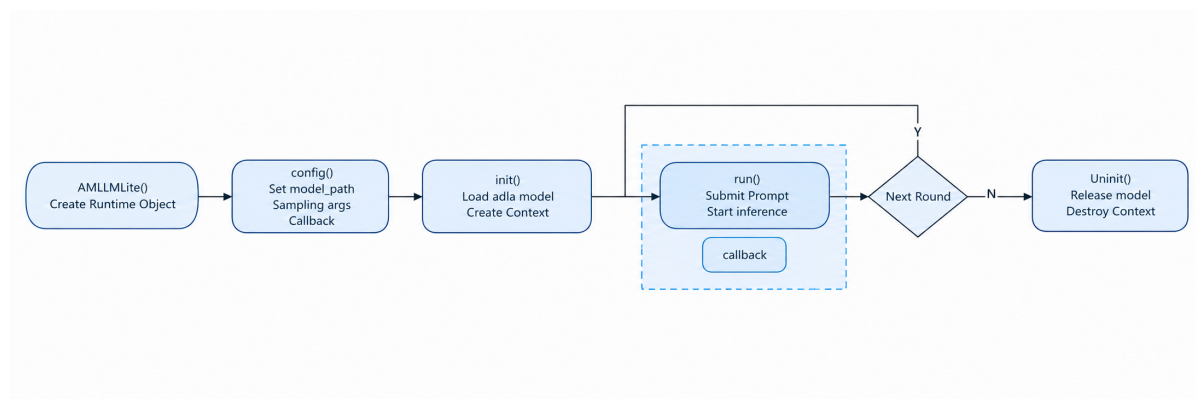
#Example
amlnn_llm.uninit()

```

4. On-Device Inference (Python API)

This chapter describes in detail the Python Runtime component `amllm-edge-toolkit-lite` of the Amlogic LLM SDK and its complete inference execution workflow on edge boards. This Python API provides a lightweight development interface for rapid prototyping and functional validation; underneath, it automatically calls the high-performance C++ Runtime to schedule the NPU for fully hardware-accelerated inference.

4.1 Workflow Overview



The end-to-end Python API call flow is shown above. Developers only need to import the `AMLLite` class from the `amllm` package to quickly develop LLM/VLM applications for the edge. The complete steps are: **Initialize the Runtime object** --> **Load the language model** --> **Load the VLM vision model (VLM-specific)** --> **Submit the inference request** --> **Stream output and aggregate results** --> **Release underlying resources uniformly**

- **Environment preparation:** Refer to Section 2.2.2.3 "Installing `amllm-edge-toolkit-lite`" to install the inference toolkit on the board.
- **Brief description of core functions**
 - **Instantiation (`AMLLite()`)**
 - Principle: creates a Runtime object in the Python process. `log_level` can be set to `DEBUG`, `INFO`, `WARNING`, or `ERROR` to control Python-level log output. At this point, only the object is created; no model is loaded.
 - **Parameter configuration (`llm.config(...)`)**
 - Principle: the Python layer collects the user-provided model path, optional `mmproj_path`, sampling parameters, and the Token callback, and stores them for passing to the underlying C++ Runtime structures (e.g., `AML_LLMLiteConfig`) at initialization. `model_path` must point to a language model `.adla` file accessible on the board; for VLM, the corresponding `mmproj_path` must also be provided. `sampling_mode` can be `argmax`, `top_p`, or `top_k`, with `top_k`, `top_p`, `temperature`, and `repeat_penalty` set as needed; `on_token` receives streamed generation results.
 - **Initialization (`llm.init()`)**
 - Principle: this is the key resource-allocation step. The Python wrapper calls the C++ Runtime's `aml_llm_init()` API. `init()` accepts no model path or sampling parameters; these must be configured in advance via `config()`. Only after initialization succeeds may `run()` be called.

- Underlying execution:
 - Model loading: the C++ Runtime loads the `.ad1a` model file from disk and performs integrity checks.
 - NPU context creation: requests hardware resources from the NPU driver, such as allocating NPU-dedicated KV Cache memory (for storing historical Key/Value states) and model weight memory.
 - Ready state: hardware abstraction layer initialization is complete; the LLM instance is ready to accept inference requests.
- **Running inference** (`llm.run(...)`)
 - Principle: starts the core computation flow of the model. The Python layer passes the user's prompt string and optional multimodal image inputs to the C++ Runtime. `prompt` is the current turn's text input; `input_type` uses `prompt` for text requests and `multimodal` for image-text requests. VLM requests must also provide RGB888 images containing `data`, `width`, and `height` in `images`, and ensure `img_content` matches the image tokens in the prompt.
 - Underlying execution:
 - Encoding and preprocessing: the C++ Runtime invokes the built-in Tokenizer to convert the input prompt text (UTF-8) into a Token ID sequence; VLM inputs additionally process the RGB888 images and the projector model.
 - Kernel scheduling: the Token ID sequence and any necessary image features are fed as model inputs, and NPU kernels are scheduled for forward computation.
 - Streaming output: after each Token is generated, the C++ Runtime fires the callback. The Python Runtime captures and processes these callbacks to provide user-friendly streaming output, and aggregates the final result when `run()` returns.
- **Resource release** (`llm.uninit()`)
 - Principle: destroys the LLM instance and cleans up NPU-occupied memory. The application should call `uninit()` after all `run()` requests and callback processing are complete; this API takes no extra parameters and safely handles repeated calls or calls before initialization.
 - Underlying execution: calls the C++ Runtime's `aml_llm_uninit()` API to release the KV Cache and weight memory occupied by the model and destroy the NPU context handle.

4.2 Python Demo Examples

4.2.1 LLM Example

Call order: create object, configure parameters, initialize, run inference, release. The callback outputs text as each Token arrives.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""Interactive AMLLLMLite demo that mirrors the C sample."""

import argparse
# argparse is used to parse model paths and sampling parameters.
import queue
```

```

# The queue safely hands requests from the input thread to the inference thread.
import sys
import threading
from amllmlite.api import AMLLLMLite
from amllmlite.api.inference import RunStatus

def stream_callback(token, userdata=None):
    # The callback fires as each Token arrives; FINISH/ERROR indicate the end or
    # failure of this generation.
    """Print tokens as they arrive (mimic C demo callback behavior)."""
    text = token.get("text", "")
    status = token.get("status")
    if userdata and not userdata.get("printed"):
        print(f"[Request #{userdata.get('request_id', 0)}]")
        userdata["printed"] = True
    if status == RunStatus.FINISH:
        print()
    elif status == RunStatus.ERROR:
        print("\n[Generation error]")
    elif text:
        print(text, end="", flush=True)

def parse_args():
    # Command-line arguments map one-to-one to AMLLLMLite.config() options.
    parser = argparse.ArgumentParser(description="Amlogic LLM interactive demo
(Python)")
    parser.add_argument("--model", required=True, help="Path to LLM model file")
    parser.add_argument("--sampling-mode", default="argmax", choices=["argmax",
"top_p", "top_k"], help="Sampling mode")
    parser.add_argument("--top-k", type=int, default=3, dest="top_k", help="Top-
K parameter")
    parser.add_argument("--top-p", type=float, default=0.9, dest="top_p",
help="Top-P parameter")
    parser.add_argument("--temperature", type=float, default=1.0, help="Softmax
temperature")
    parser.add_argument("--repeat-penalty", type=float, default=1.1,
dest="repeat_penalty", help="Repeat penalty factor")
    parser.add_argument("--log-level", default="ERROR", dest="log_level",
choices=["DEBUG", "INFO", "WARNING", "ERROR"])
    return parser.parse_args()

def main():
    # Standard lifecycle: create, config, init, run loop, uninit in finally.
    args = parse_args()
    amllm = AMLLLMLite(log_level=args.log_level)
    # config must be called before init; the model path must point to an ADLA
    file accessible on the board.
    amllm.config(
        model_path=args.model,
        sampling_mode=args.sampling_mode,
        top_k=args.top_k,
        top_p=args.top_p,

```

```

        temperature=args.temperature,
        repeat_penalty=args.repeat_penalty,
        on_token=stream_callback,
    )
    amlllm.init()

    print("Welcome to Amlogic LLM interactive demo (Python).")
    print("Commands: exit | new_talk | break")

    user_state = {"request_id": 0, "printed": False}

    input_queue = queue.Queue()

    def read_input():
        while True:
            try:
                user_input = input().strip()
            except EOFError:
                input_queue.put("exit")
                break
            if user_input == "break":
                try:
                    amlllm.break_generation()
                    print("\nstop signal sent.")
                except Exception as exc:
                    print(f"\nBreak failed: {exc}")
                    continue
            if user_input == "new_talk":
                try:
                    amlllm.reset_session()
                    print("\nConversation state cleared.")
                except Exception as exc:
                    print(f"\nReset failed: {exc}")
                    continue
            input_queue.put(user_input)
            if user_input == "exit":
                break

    input_thread = threading.Thread(target=read_input, daemon=True)
    input_thread.start()

    try:
        while True:
            print("\nLLM@Amlogic>>> ", end="", flush=True)
            user_input = input_queue.get()
            if not user_input:
                print("Please enter a non-empty prompt.")
                continue
            if user_input == "exit":
                break

            try:
                user_state["request_id"] += 1
                user_state["printed"] = False

```

```

# prompt mode requires no images; run returns aggregated text and
Token statistics.
result = amlllm.run(
    prompt=user_input,
    input_type="prompt",
    run_mode="generate",
    retain_history=False,
    enable_think=False,
    role="user",
    user_data=user_state,
)
if not result["text"].endswith("\n"):
    print()
print(f"Tokens generated: {result['token_count']}")
except KeyboardInterrupt:
    print("\nKeyboardInterrupt received. Sending break...")
    amlllm.break_generation()
except Exception as exc:
    print(f"\nGeneration failed: {exc}")
finally:
    amlllm.uninit()

if __name__ == "__main__":
    sys.exit(main())

```

`retain_history` is explicitly set to `False` in this example. For multi-turn history, set it to `True` or pass it per request in the application code; typing `new_talk` clears the session, and typing `break` interrupts the current generation.

4.2.2 VLM Example

Call order: create object, configure parameters, initialize, run inference, release. VLM configuration adds `mmproj_path`, and images are passed at runtime. Pillow is only responsible for decoding image files into tightly packed RGB888 bytes; it is not a required Runtime dependency.

```

#!/usr/bin/env python3
"""Interactive AMLLMlite VLM demo corresponding to LLM SDK main_vlm.cpp."""

import argparse
# argparse is used to parse model paths and sampling parameters.
import sys

from amllmlite.api import AMLLMlite
from amllmlite.api.inference import RunStatus

def stream_callback(token):
    status = token.get("status")
    if status == RunStatus.FINISH:
        print()
    elif status == RunStatus.ERROR:
        print("\n[Generation error]")
    elif token.get("text"):

```

```

        print(token["text"], end="", flush=True)

def load_rgb_image(path):
    # Convert any Pillow-supported format into tightly packed RGB data.
    try:
        from PIL import Image
        # Pillow handles image decoding; the Runtime itself only accepts RGB888
        bytes.
    except ImportError as exc:
        raise RuntimeError("VLM demo requires Pillow: pip install Pillow") from
    exc

    with Image.open(path) as image:
        rgb = image.convert("RGB")
        width, height = rgb.size
        return {"data": rgb.tobytes(), "width": width, "height": height}

def parse_args():
    # Command-line arguments map one-to-one to AMLLLMLite.config() options.
    parser = argparse.ArgumentParser(description="Amlogic VLM interactive demo")
    parser.add_argument("--model", required=True, help="Path to LLM model file")
    parser.add_argument("--mmproj", required=True, help="Path to multimodal
    projector model file")
    parser.add_argument("--sampling-mode", default="argmax", choices=["argmax",
    "top_p", "top_k"])
    parser.add_argument("--top-k", type=int, default=3, dest="top_k", help="Top-
    K parameter")
    parser.add_argument("--top-p", type=float, default=0.9, dest="top_p",
    help="Top-P parameter")
    parser.add_argument("--temperature", type=float, default=1.0, help="Softmax
    temperature")
    parser.add_argument("--repeat-penalty", type=float, default=1.1,
    dest="repeat_penalty", help="Repeat penalty factor")
    parser.add_argument("--log-level", default="ERROR", dest="log_level",
    choices=["DEBUG", "INFO", "WARNING", "ERROR"])
    parser.add_argument("--img-start", default="<|vision_start|>")
    parser.add_argument("--img-end", default="<|vision_end|>")
    parser.add_argument("--img-content", default="<image>")
    return parser.parse_args()

def main():
    # Standard lifecycle: create, config, init, run loop, uninit in finally.
    args = parse_args()
    amlllm = AMLLLMLite(log_level=args.log_level)
    # config must be called before init; the model path must point to an ADLA
    file accessible on the board.
    amlllm.config(
        model_path=args.model,
        mmproj_path=args.mmproj,
        sampling_mode=args.sampling_mode,
        top_k=args.top_k,
        top_p=args.top_p,

```

```

        temperature=args.temperature,
        repeat_penalty=args.repeat_penalty,
        on_token=stream_callback,
    )
    amlllm.init()
    # images holds images queued for the next turn; it is cleared after
inference.
    images = []

    print("VLM model loaded. Use /image <path> to append images, then enter a
prompt.")
    print("Commands: exit | new_talk | break")
    try:
        while True:
            try:
                line = input("\nVLM>>> ").strip()
            except EOFError:
                break
            if not line:
                print("Please enter a non-empty prompt.")
                continue
            if line == "exit":
                break
            if line == "new_talk":
                amlllm.reset_session()
                images.clear()
                print("Conversation state cleared.")
                continue
            if line == "break":
                amlllm.break_generation()
                print("Stop signal sent.")
                continue

            # /image only loads and stages the image; it does not trigger
inference immediately.
            if line.startswith("/image "):
                path = line[7:].strip()
                try:
                    image = load_rgb_image(path)
                except Exception as exc:
                    print(f"Image load failed: {exc}")
                    continue
                images.append(image)
                print(
                    f"Image appended: {path},
{image['width']}x{image['height']}, "
                    f"total={len(images)}"
                )
                continue

            prompt = line
            if images and args.img_content not in prompt:
                prompt = args.img_content * len(images) + prompt
                print(f"Prepended {len(images)} image marker(s).")

            print("robot: ", end="", flush=True)

```

```

# multimodal mode requires mmproj_path, images, and image markers to
all be valid.
result = amlllm.run(
    prompt=prompt,
    input_type="multimodal" if images else "prompt",
    retain_history=False,
    enable_think=False,
    role="user",
    images=images or None,
    img_start=args.img_start,
    img_end=args.img_end,
    img_content=args.img_content,
)
if not result["text"].endswith("\n"):
    print()
images.clear()
finally:
    amlllm.uninit()
return 0

if __name__ == "__main__":
    sys.exit(main())

```

Entering `/image <path>` stages one or more images; the next plain-text input uses those images and clears the staged list after inference. `new_talk` clears both the Runtime session and the image list. If the prompt contains no `<image>`, the demo automatically prepends as many markers as there are images. To enable cross-turn history, change `retain_history=False` to `True` in both demos.

4.3 Python API Reference

All of the following APIs belong to `amllmlite.api.AMLLMLite`. Except for `__init__`, the APIs should be used in the order "`config()` → `init()` → `run()` → `uninit()`"; the Runtime raises an exception if prerequisites are not met.

4.3.1 Initialization

Function: Creates the Python Runtime object and sets the Python-side log level; no model is loaded at this point.

Parameter	Type	Description
<code>log_level</code>	<code>str</code>	Log level: <code>DEBUG</code> , <code>INFO</code> , <code>WARNING</code> , or <code>ERROR</code> .

Return value: an `AMLLMLite` instance.

```

from amllmlite.api import AMLLLMLite

amlllm = AMLLLMLite(log_level="DEBUG")

```

4.3.2 setLogLevel

Function: Updates the log level at runtime.

Parameter	Type	Description
level	str	One of <code>DEBUG</code> , <code>INFO</code> , <code>WARNING</code> , <code>ERROR</code> .

Return value: none.

4.3.3 config

Function: Stores the model path, VLM projector, sampling policy, and Token callback, and creates the underlying Runtime; must be called before `init()`. Calling it again closes and rebuilds the existing Runtime.

Parameter	Type	Description
model_path	str	The LLM or VLM language model <code>.adla</code> file; the path must exist.
mmproj_path	str / None	Multimodal projector <code>.adla</code> file; leave empty for pure LLM.
sampling_mode	str	<code>argmax</code> , <code>top_p</code> , or <code>top_k</code> (alias <code>topk</code> accepted).
top_k	int	Top-K candidate count; must be greater than 0.
top_p	float	Top-P cumulative probability; range <code>(0, 1]</code> .
temperature	float	Softmax temperature; must be greater than 0.
repeat_penalty	float	Repeat penalty factor; must be greater than 0.
on_token	callable / None	Callback fired once per Token; signature may be <code>(token)</code> or <code>(token, user_data)</code> .

Return value: none. Raises `ValueError` or `FileNotFoundError` for invalid paths, sampling parameters, or callbacks.

4.3.4 init

Function: Loads the ADLA model, creates the NPU context, and allocates resources required for inference.

Return value: none; raises `RuntimeError` or an underlying exception if initialization fails.

4.3.5 run

Function: Performs one text or multimodal generation and streams results via `on_token`.

Parameter	Type	Description
prompt	str	Non-empty UTF-8 prompt.

Parameter	Type	Description
<code>input_type</code>	<code>str</code>	<code>prompt</code> (text only) or <code>multimodal</code> (image + text).
<code>run_mode</code>	<code>str</code>	Currently only <code>generate</code> is supported.
<code>retain_history</code>	<code>bool</code>	Whether to keep this turn's KV Cache; set to <code>True</code> for multi-turn dialogue.
<code>enable_think</code>	<code>bool</code>	Whether to enable the model's thinking mode.
<code>role</code>	<code>str</code>	Message role used by the Chat Template; must not be empty.
<code>images</code>	<code>list / None</code>	VLM image list; each item is <code>{"data": bytes, "width": int, "height": int}</code> , where <code>data</code> is tightly packed RGB888 bytes.
<code>img_start</code>	<code>str</code>	Image start marker.
<code>img_end</code>	<code>str</code>	Image end marker.
<code>img_content</code>	<code>str</code>	Image content marker; must match the marker in the prompt.
<code>user_data</code>	<code>object / None</code>	User data passed through to the Token callback.

Multimodal runs require `mmproj_path` to be provided in `config()` and `images` to contain at least one image; the number of images and the `img_content` markers in the prompt should match.

Return value: a dict containing `text` (full text), `token_ids` (list of generated Token IDs), `token_count` (number generated), and `status` (a `RunStatus` state).

4.3.6 `reset_session`

Function: Clears the KV Cache and history state of the current session without unloading the model; suitable for starting a new conversation.

Return value: none.

4.3.7 `break_generation`

Function: Requests interruption of the currently running generation.

Return value: none. Upon receiving the signal, the generation thread returns the final state via the callback.

4.3.8 `set_chat_template`

Function: Sets the system prompt and the user-message prefix/suffix of the native Chat Template.

Parameter	Type	Description
-----------	------	-------------

Parameter	Type	Description
<code>system_prompt</code>	<code>str</code>	System message content.
<code>prompt_prefix</code>	<code>str</code>	Text prepended to each user prompt.
<code>prompt_postfix</code>	<code>str</code>	Text appended after each user prompt.

Return value: none.

4.3.9 `uninit`

Function: Stops the Runtime and releases the model, KV Cache, NPU context, and other resources.

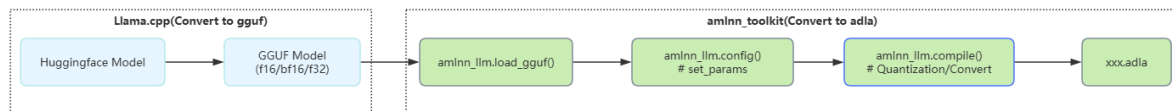
Return value: none. Recommended to call inside `try...finally`; it safely skips if not initialized.

5. On-Device Inference (C++ API)

This chapter describes the C++ API specifications and call flow of LLMSDK in detail. All API declarations are in the header file `llmsdk.h`. To ensure correct resource management and stable inference, developers must strictly follow this lifecycle order: **define callbacks and configuration** → **initialize the model** → **construct inputs and run inference** → **session management (retain/reset)** → **release the context**.

Note: For the validity and lifetime management of all pointer parameters, strictly follow the specific descriptions of each API.

5.1 Workflow Overview



1. Core call flow

1. **Preparation:** configure the model path, sampling parameters, and the result callback function.
2. **Initialization:** call `aml_llm_init()` to create the runtime context.
3. **Inference:** construct input data and submit the inference request.
4. **Session management:** retain the historical context or reset the session according to business needs.
5. **Resource release:** call `aml_llm_uninit()` to clean up all resources.

Key constraints:

- The context handle spans the entire lifecycle and must be properly stored.
- After failed initialization or after resources have been released, it is strictly forbidden to continue calling any API with that handle.
- Inference operations may only be performed after `aml_llm_init()` has returned successfully.

2. Model types and input handling

The SDK supports both text-only LLM and multimodal VLM scenarios. They share the same runtime APIs, but differ in initialization configuration and input construction.

1. Text-only LLM

- **Input method:** pass the prompt text directly.
- **Processing mechanism:** the SDK internally combines the Chat Template to automatically perform formatting and tokenization; suitable for regular dialogue scenarios.

2. Vision Language Model (VLM)

- **Initialization configuration:** in addition to the base model path, the **Projector model path** must be specified in the extended configuration.
- **Input construction:**
 - **Image data:** provide raw RGB888 image data.
 - **Text prompt:** provide a prompt containing image tokens (Image Tokens).
- **Consistency requirements:**

- **Count consistency:** the number of submitted images must equal the number of image tokens in the prompt.
- **Order consistency:** the ordering of the image data must strictly correspond to the order in which the tokens appear in the prompt.
- **Purpose:** ensure the SDK maps the visual embeddings to the correct positions in the text sequence, achieving image-text alignment.

3. Asynchronous inference and callback mechanism

Inference results are returned asynchronously via the registered `LLMResultCallback` callback, supporting streaming output and status monitoring.

- **Callback trigger:** the SDK fires the callback for each generated token or whenever the inference status changes.
- **Application handling:** within the callback, developers can:
 - Accumulate generated text fragments.
 - Update the UI in real time for streaming display.
 - Trigger follow-up business logic (e.g., interrupting generation, tool calls, etc.).

4. Resource release

When all inference tasks are complete and callback processing has finished, **you must** call `aml_llm_uninit()` to release resources.

Released resources include:

- Model weight memory.
- KV Cache storage.
- Inference thread resources.
- NPU hardware context handle.

Note: It is recommended to call this API explicitly before application exit or module unload, to avoid memory leaks or NPU resource occupation that could cause subsequent load failures.

5.2 Core Data Structures and Enums

This section describes the handles, enums, and structures as defined in `llmsdk.h`.

5.2.1 LLMContext

The type `LLMContext` represents the model runtime handle; its actual type is `void*`. The handle is output by `aml_llm_init()` and passed to other APIs; it becomes invalid after `aml_llm_uninit()` is called.

```
typedef void* LLMContext;
```

5.2.2 AML_LLMRunStatus

The enum `AML_LLMRunStatus` represents the run status of a generation task as reported in the callback.

Callback status enum:

Member	Description
<code>AML_LLM_RUN_NORMAL</code>	Inference in progress; this callback carries one generation result.
<code>AML_LLM_RUN_FINISH</code>	Inference finished, usually meaning EOS was encountered or the generation limit was reached.
<code>AML_LLM_RUN_ERROR</code>	An inference error occurred.

5.2.3 AML_LLMRetStatus

The enum `AML_LLMRetStatus` indicates whether an API call succeeded.

API return statuses:

Member	Description
<code>AML_LLM_Status_Success</code>	The call succeeded. For asynchronous inference, it means the request was accepted.
<code>AML_LLM_Status_Failed</code>	The call failed, possibly due to an invalid handle, parameter, model, or device resource.

5.2.4 AML_LLMInputType

The enum `AML_LLMInputType` indicates the input data type used by `AML_LLMInput`.

Member	Description
<code>AML_LLM_INPUT_PROMPT</code>	UTF-8 text; the SDK automatically applies the Chat Template and tokenizes.
<code>AML_LLM_INPUT_MULTIMODAL</code>	Text plus RGB888 image/video frames; the SDK performs vision processing.
<code>AML_LLM_INPUT_TOKEN</code>	Token IDs that have already been templated and tokenized; the SDK does not modify them.

5.2.5 AML_LLMRunMode

The enum `AML_LLMRunMode` indicates the inference mode of the current run.

Member	Description
<code>AML_LLM_RUN_GENERATE</code>	Text generation mode.
<code>AML_LLM_RUN_EMBEDDING</code>	Embedding generation mode; reserved in the current version.

5.2.6 AML_LLMImageType

The enum `AML_LLMImageType` indicates the storage form of VLM image data.

Member	Description
<code>AML_LLM_IMAGE_TYPE_BUFFER</code>	Tightly packed RGB888 image buffer.

5.2.7 AML_LLMSamplingMode

The enum `AML_LLMSamplingMode` indicates the sampling strategy used for text generation.

Member	Description
<code>AML_LLM_ARG_Max</code>	Argmax: selects the highest-probability token; deterministic and reproducible.
<code>AML_LLM_TOP_P</code>	Top-P nucleus sampling; $0 < \text{top_p} \leq 1$.
<code>AML_LLM_TOP_K</code>	Top-K sampling; $\text{top_k} > 0$.
<code>AML_LLM_CHAIN_SAMPLER</code>	Samples according to a JSON sampler chain configuration, provided in <code>sampler_params_json</code> .

5.2.8 AML_LLMInitConfig

The struct `AML_LLMInitConfig` represents the initialization configuration needed to create an LLM/VLM runtime context.

Initialization configuration struct:

Member	Type	Description
<code>model_path</code>	<code>const char*</code>	Path to the language model ADLA file. Must remain valid during the call.
<code>sampling_mode</code>	<code>AML_LLMSamplingMode</code>	Default sampling policy.
<code>top_k</code>	<code>int32_t</code>	Top-K parameter; must be > 0 when TOP_K is used.
<code>top_p</code>	<code>float</code>	Top-P parameter; must be in $(0,1]$ when TOP_P is used.
<code>temperature</code>	<code>float</code>	Temperature coefficient; must be greater than 0.
<code>repeat_penalty</code>	<code>float</code>	Repeat penalty factor; must be greater than 0; 1.0 means no penalty.
<code>init_extend</code>	<code>AML_LLMInitExtend</code>	Extended parameters.

5.2.9 AML_LLMInitExtend

The struct `AML_LLMInitExtend` represents optional extended parameters for the initialization stage.

Member	Type	Description
<code>mmproj_path</code>	<code>const char*</code>	Path to the VLM projector ADLA; pass <code>NULL</code> for text-only models.
<code>sampler_params_json</code>	<code>const char*</code>	JSON configuration for <code>AML_LLM_CHAIN_SAMPLER</code> ; pass <code>NULL</code> or an empty string when unused.
<code>reserved</code>	<code>uint8_t[1024-...]</code>	Reserved fields; must be zeroed.

5.2.10 AML_LLMRunConfig

The struct `AML_LLMRunConfig` represents the run mode, history retention, and runtime extended configuration of one inference request.

Member	Type	Description
<code>run_mode</code>	<code>AML_LLMRunMode</code>	Usually set to <code>AML_LLM_RUN_GENERATE</code> .
<code>retain_history</code>	<code>int</code>	Set to 1 to retain the KV Cache and dialogue/vision history; set to 0 to not retain.
<code>enable_think</code>	<code>int</code>	Set to 1 to enable thinking mode; whether it takes effect depends on the model template.
<code>run_extend</code>	<code>AML_LLMRunExtend</code>	Extended parameters.

5.2.11 AML_LLMRunExtend

The struct `AML_LLMRunExtend` represents per-inference sampling parameter overrides.

Member	Type	Description
<code>sampling_config_valid</code>	<code>uint32_t</code>	Set to 1 to use all sampling fields of this struct; set to 0 to use the initialization defaults.
<code>sampling_mode</code>	<code>AML_LLMSamplingMode</code>	Sampling policy for this run.
<code>top_k / top_p</code>	<code>int32_t / float</code>	Top-K/Top-P parameters for this run.
<code>temperature / repeat_penalty</code>	<code>float / float</code>	Temperature and repeat penalty for this run.
<code>sampler_params_json</code>	<code>const char*</code>	Chain Sampler JSON for this run; pass <code>NULL</code> when unused.

Member	Type	Description
<code>reserved</code>	<code>uint8_t[...]</code>	Reserved fields; must be zeroed.

5.2.12 AML_LLMImageInput

The struct `AML_LLMImageInput` represents one frame of raw RGB888 image data passed to a VLM.

Member	Type	Description
<code>type</code>	<code>AML_LLMImageType</code>	Currently only <code>AML_LLM_IMAGE_TYPE_BUFFER</code> is supported.
<code>data</code>	<code>const void*</code>	Tightly packed RGB888 data, size <code>width*height*3</code> , with no stride, row padding, or file header.
<code>width / height</code>	<code>uint32_t</code>	Original image width/height in pixels.
<code>timestamp</code>	<code>uint64_t</code>	Video frame timestamp in milliseconds; 0 for ordinary images.

The image buffer must remain valid until `aml_llm_run()` returns; compressed data such as JPEG/PNG must first be decoded to RGB888 by the application.

5.2.13 AML_LLMMultimodalInput

The struct `AML_LLMMultimodalInput` represents the text prompt and image array of one VLM request.

Member	Type	Description
<code>prompt</code>	<code>const char*</code>	UTF-8 prompt, with <code>img_content</code> markers placed at image positions.
<code>img_inputs</code>	<code>const AML_LLMImageInput*</code>	Array of images/video frames. The order must match the order of the markers in the prompt.
<code>img_count</code>	<code>uint32_t</code>	Number of elements in the array; when greater than 0, the array must be valid.
<code>img_start / img_end</code>	<code>const char*</code>	Boundary markers around the image embedding; usually empty for FastVLM/MLP.
<code>img_content</code>	<code>const char*</code>	Image placeholder in the prompt; defaults to <code><image></code> when NULL.
<code>sample_rate_num / sample_rate_den</code>	<code>uint32_t</code>	Reserved video sample rate fields; not used by the current Runtime.

5.2.14 AML_LLMTokenInput

The struct `AML_LLMTokenInput` represents token ID inputs that have already been templated and tokenized.

Member	Type	Description
<code>input_ids</code>	<code>const int32_t*</code>	Final token ID array; the caller is responsible for templating, roles, and tokenization.
<code>n_tokens</code>	<code>int32_t</code>	Number of tokens; must be greater than 0.

5.2.15 AML_LLMInput

The struct `AML_LLMInput` represents the unified input object passed to `aml_llm_run()`.

Member	Type	Description
<code>input_type</code>	<code>AML_LLMInputType</code>	Determines which union member is used.
<code>prompt_input</code>	<code>const char*</code>	Text for the <code>PROMPT</code> type.
<code>multimodal_input</code>	<code>AML_LLMMultimodalInput</code>	Input for the <code>MULTIMODAL</code> type.
<code>token_input</code>	<code>AML_LLMTokenInput</code>	Input for the <code>TOKEN</code> type.
<code>role</code>	<code>const char*</code>	Message role; defaults to <code>user</code> when NULL.

5.2.16 AML_LLMGenerationResult

The struct `AML_LLMGenerationResult` represents one incremental output in text generation mode, usually corresponding to one generated token.

Member	Type	Description
<code>text</code>	<code>const char*</code>	Current UTF-8 text fragment, valid only during the callback.
<code>token_id</code>	<code>int32_t</code>	ID of the currently generated token.

5.2.17 AML_LLMEmbeddingResult

The struct `AML_LLMEmbeddingResult` represents embedding data returned in embedding mode; this mode is reserved in the current version.

Member	Type	Description
<code>values</code>	<code>const float*</code>	Embedding float array, valid only during the callback.
<code>dimension</code>	<code>uint32_t</code>	Number of float elements in <code>values</code> .

5.2.18 AML_LLMResult

The struct `AML_LLMResult` represents the result of one inference callback. In actual use, select the `generation` or `embedding` member according to `AML_LLMRunConfig.run_mode`; the contents pointed to by the pointers are valid only during the callback.

Member	Type	Description
<code>generation.text</code>	<code>const char*</code>	Current UTF-8 text fragment, valid only during the callback.
<code>generation.token_id</code>	<code>int32_t</code>	Current token ID.
<code>embedding.values</code>	<code>const float*</code>	Embedding data; currently a reserved feature.
<code>embedding.dimension</code>	<code>uint32_t</code>	Embedding dimension.

5.3 Basic API Reference

This section describes the result callback and the core APIs for model initialization, inference, and resource release. The callback is registered by the application and invoked by the SDK per token or per status change during generation.

5.3.1 LLMResultCallback

The function type `LLMResultCallback` defines the parameters and calling convention of the inference result callback. The SDK returns results via `result`, passes the application context verbatim via `userdata`, and reports whether this inference is in progress, finished, or errored via `state`.

```
typedef void (*LLMResultCallback)(AML_LLMResult *result, void *userdata,
                                   AML_LLMRunStatus state);
```

Parameter	Type	Description
<code>result</code>	<code>AML_LLMResult*</code>	Current generation result; may be <code>NULL</code> when the status is error.
<code>userdata</code>	<code>void*</code>	User data pointer passed in by the application.
<code>state</code>	<code>AML_LLMRunStatus</code>	Run status of this callback.

Data should be processed quickly in the callback; if `text` or `values` must be used asynchronously, the content must be copied.

5.3.2 aml_llm_init

The API `aml_llm_init` loads the language model and the optional VLM projector, creates the runtime context, and registers the result callback.

API	<code>aml_llm_init</code>
Function	Loads the language model (and the optional VLM projector), creates the <code>LLMContext</code> , and registers the result callback.
Parameters	<code>LLMContext* context</code> : output context handle. <code>AML_LLMInitConfig* init_config</code> : initialization configuration. <code>LLMResultCallback callback</code> : streaming result callback; may be <code>NULL</code> .
Return value	<code>AML_LLM_Status_Success</code> : initialization succeeded. <code>AML_LLM_Status_Failed</code> : model, parameter, device, or memory error.

Example:

```
LLMContext context = NULL;
AML_LLMInitConfig cfg = {0};
cfg.model_path = "model.ad1a";
cfg.sampling_mode = AML_LLM_ARG_Max;
cfg.top_k = 3; cfg.top_p = 0.9f;
cfg.temperature = 1.0f; cfg.repeat_penalty = 1.1f;
AML_LLMRetStatus ret = aml_llm_init(&context, &cfg, callback);
```

5.3.3 aml_llm_run

The API `aml_llm_run` submits one text, token, or multimodal inference request; generation results are returned via the callback registered at initialization, token by token.

API	<code>aml_llm_run</code>
Function	Executes one inference and triggers the callback asynchronously; when <code>retain_history=1</code> , the session KV Cache is retained.
Parameters	<code>LLMContext context</code> : valid context. <code>AML_LLMInput* input</code> : this run's input. <code>AML_LLMRunConfig* run_config</code> : run mode, history, and sampling override configuration. <code>void* userdata</code> : callback user data, returned by the SDK as-is.
Return value	<code>AML_LLM_Status_Success</code> : the request was accepted. <code>AML_LLM_Status_Failed</code> : invalid context, input, or run configuration.

Example:

```
AML_LLMInput input = {0};
input.input_type = AML_LLM_INPUT_PROMPT;
input.prompt_input = "你好，请介绍一下自己。";
AML_LLMRunConfig run = {0};
run.run_mode = AML_LLM_RUN_GENERATE;
run.retain_history = 1;
AML_LLMRetStatus ret = aml_llm_run(context, &input, &run, userdata);
```

5.3.4 aml_llm_uninit

The API `aml_llm_uninit` releases the model, KV Cache, thread, and context resources.

API	<code>aml_llm_uninit</code>
Function	Unloads the model and destroys the context. The handle must not be used again after this call.
Parameters	<code>LLMContext context</code> : valid handle returned by <code>aml_llm_init()</code> ; should be called after all callbacks have completed.
Return value	<code>AML_LLM_Status_Success</code> : release succeeded. <code>AML_LLM_Status_Failed</code> : invalid handle or release failure.

Example:

```
aml_llm_uninit(context);  
context = NULL;
```

5.3.5 aml_llm_reset

The API `aml_llm_reset` clears the KV Cache, text history, and visual embeddings of the current session, returning the context to a state where a new session can begin.

API	<code>aml_llm_reset</code>
Function	Clears the KV Cache, text history, and visual embeddings so the context starts a new session.
Parameters	<code>LLMContext context</code> : valid context; if a run is in progress, call <code>aml_llm_break()</code> first and wait for the callback to finish.
Return value	<code>AML_LLM_Status_Success</code> : reset succeeded. <code>AML_LLM_Status_Failed</code> : invalid handle or reset failed.

Example:

```
AML_LLMRetStatus ret = aml_llm_reset(context);
```

5.3.6 aml_llm_break

The API `aml_llm_break` requests interruption of the currently executing generation task; the task's final status is still notified via `LLMResultCallback`.

API	<code>aml_llm_break</code>
Function	Requests interruption of the current generation task; the final status is still notified via the callback.
Parameters	<code>LLMContext context</code> : valid context with inference in progress.

API	<code>aml_llm_break</code>
Return value	<code>AML_LLM_Status_Success</code> : break request submitted. <code>AML_LLM_Status_Failed</code> : invalid handle or submission failed.

Example:

```
aml_llm_break(context);
```

5.3.7 aml_llm_set_chat_template

The API `aml_llm_set_chat_template` sets the system prompt and the user input prefix/suffix templates used for subsequent prompt inputs.

API	<code>aml_llm_set_chat_template</code>
Function	Sets the system prompt, user input prefix, and suffix, overriding the template fragments for subsequent prompt requests. Token inputs do not pass through this template.
Parameters	<code>LLMContext context</code> : valid context. <code>const char* system_prompt</code> : system prompt; may be NULL. <code>const char* prompt_prefix</code> : user input prefix. <code>const char* prompt_postfix</code> : user input suffix.
Return value	<code>AML_LLM_Status_Success</code> : set successfully. <code>AML_LLM_Status_Failed</code> : invalid handle or string parameters.

Example:

```
aml_llm_set_chat_template(context,
                          "<|im_start|>system\nYou are a helpful assistant.",
                          "<|im_end|>\n",
                          "<|im_start|>user\n",
                          "<|im_end|>\n<|im_start|>assistant\n");
```

5.4 Error Return Codes

The current LLMSDK APIs use a unified binary return code:

Return value	Description
<code>AML_LLM_Status_Success</code>	The API call succeeded; for asynchronous inference it means the request was submitted.
<code>AML_LLM_Status_Failed</code>	The API call failed. Diagnose the specific cause via logs and parameter checks, including invalid context, model load failure, illegal input, out-of-range sampling parameters, insufficient memory, or device unavailability.

5.5 Multimodal Notes

LLMSDK supports VLM image-text inference via `AML_LLM_INPUT_MULTIMODAL`. The language model and projector must be converted into ADLA files separately, with the projector specified in `AML_LLMInitExtend.mmproj_path`. The SDK handles image scaling, normalization, vision encoding, and embedding injection; the application is responsible for image decoding and buffer management.

5.5.1 Multimodal Call Flow

1. Set the language model path and `init_extend.mmproj_path` in `AML_LLMInitConfig`.
2. Decode the compressed image into a tightly packed RGB888 buffer.
3. Construct `AML_LLMImageInput` and `AML_LLMMultimodalInput` so that the image array order matches the `<image>` markers in the prompt.
4. Set `input.input_type = AML_LLM_INPUT_MULTIMODAL` and call `aml_llm_run()`.
5. Set `retain_history` to 1 for the first turn; subsequent text-only requests can reuse the image embeddings and KV Cache.
6. Call `aml_llm_reset()` for a new session; call `aml_llm_uninit()` when finished.

5.5.2 VLM Example

```
AML_LLMInitConfig cfg = {0};
cfg.model_path = "fastvlm.adla";
cfg.sampling_mode = AML_LLM_ARG_Max;
cfg.top_k = 3; cfg.top_p = 0.9f;
cfg.temperature = 1.0f; cfg.repeat_penalty = 1.1f;
cfg.init_extend.mmproj_path = "mmproj.adla";
LLMContext ctx = NULL;
aml_llm_init(&ctx, &cfg, callback);

AML_LLMImageInput image = {0};
image.type = AML_LLM_IMAGE_TYPE_BUFFER;
image.data = rgb888_data;
image.width = width; image.height = height;
AML_LLMInput input = {0};
input.input_type = AML_LLM_INPUT_MULTIMODAL;
input.role = "user";
input.multimodal_input.prompt = "<image>Describe this image";
input.multimodal_input.img_inputs = &image;
input.multimodal_input.img_count = 1;
input.multimodal_input.img_content = "<image>";
AML_LLMRunConfig run = {0};
run.run_mode = AML_LLM_RUN_GENERATE;
run.retain_history = 1;
aml_llm_run(ctx, &input, &run, NULL);

/* When retaining history, plain-text questions can be sent afterwards */
AML_LLMInput followup = {0};
followup.input_type = AML_LLM_INPUT_PROMPT;
followup.prompt_input = "还有其他内容吗? ";
aml_llm_run(ctx, &followup, &run, NULL);
```

For video scenarios, the application extracts frames using components such as FFmpeg and passes them in chronological order; `timestamp` uses milliseconds. Image data and strings must remain valid until `aml_llm_run()` returns.

6. FAQ and Performance Reference

6.1 Performance Reference Data

For detailed performance test data of LLMSDK on different models and hardware platforms, including throughput (Tokens/sec) and latency metrics, please refer to the following GitHub link:

Performance reference data: Amlogic LLMSDK Performance Reference Data

6.2 Common Error Troubleshooting

Question 1: Can customers' own models be supported?

Answer: If the customer's model is fine-tuned from a model in the Model_list in Chapter 1, conversion and deployment are currently supported. Other LLMs are not yet supported; more LLM models will be released in subsequent versions.

6.3 Reference Links

[https://github.com/Amlogic-NN/aamlnn-toolkit/tree/main/aamlnn_toolkit_lite/aamlm_edge_toolkit_lite](https://github.com/Amlogic-NN/amlnn-toolkit/tree/main/aamlnn_toolkit_lite/aamlm_edge_toolkit_lite)

<https://github.com/Amlogic-NN/aamlnn-model-playground/tree/main/examples/LLMs>